SET 7

PART A — Short questions and answers

Q1. Explain a derived attribute using Years_Of_Service of an EMPLOYEE.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q2. Explain the purpose of the ALTER TABLE command with an example of adding a column.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q3. Summarize the condition for a relation to be in Second Normal Form (2NF).

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q4. Compare a subquery and a join for retrieving the employees of the Sales department.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q5. Explain how MAX and MIN find the highest and lowest BasicPay.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q6. Describe how a value is assigned to a variable using the := operator in PL/SQL.

Answer: PL/SQL adds procedural programming to SQL. A procedure is a reusable named block that can take parameters.

Q7. Explain how the data of an EMPLOYEE table row maps to a MongoDB document.

Answer: A MongoDB document is one record represented as field–value pairs, similar to a JSON object.

Q8. Explain an embedded document using an Address field.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q9. Interpret the MongoDB query: db.employees.find({$or: [{Designation: "Manager"}, {BasicPay: {$gt: 60000}}]})

Answer: The query returns all documents that satisfy the specified filter condition.

Q10. Explain the purpose of the MERGE clause in Cypher.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

PART B — System solution

Scenario: A software company maintains a Employee Payroll System to store department details, employee details,

1(a) ER design:

• **DEPARTMENT**(DeptID, DeptName, Location); primary key: **DeptID**.

• **EMPLOYEE**(EmpID, EmpName, Designation, DeptID, JoiningDate, BasicPay); primary key: **EmpID**.

• **SALARY**(SalaryID, EmpID, PayMonth, Allowances, Deductions); primary key: **SalaryID**.

Connect the two main entity tables through the transaction/relationship table. The transaction table has its own primary key and foreign keys to both entity tables. Cardinality: each entity can participate in many transactions; each transaction belongs to one row from each entity table (1:M on both sides, resolving M:N). Include the transaction's date/status/amount attributes shown in the question.

1(b) SQL DDL/DML: Create each table with its listed columns, PRIMARY KEY on its ID, and FOREIGN KEYs in the transaction table. Add NOT NULL to required fields; use CHECK/UNIQUE/DEFAULT where the question asks. Insert five sample rows per table, making sure foreign-key values exist.

```
CREATE TABLE DEPARTMENT (
  DeptID NUMBER PRIMARY KEY,
  DeptName VARCHAR2(100),
  Location VARCHAR2(100)
);
CREATE TABLE EMPLOYEE (
  EmpID NUMBER PRIMARY KEY,
  EmpName VARCHAR2(100),
```

```
  Designation VARCHAR2(100),
  DeptID NUMBER,
  JoiningDate DATE,
  BasicPay VARCHAR2(100)
);
CREATE TABLE SALARY (
  SalaryID NUMBER PRIMARY KEY,
  EmpID NUMBER,
  PayMonth VARCHAR2(100),
  Allowances VARCHAR2(100),
  Deductions VARCHAR2(100)
);
```

2(a) Join query: Select the two entity names/details from the transaction table by joining the transaction table to both entity tables using their IDs.

```
SELECT a.*, b.*
FROM SALARY x
JOIN DEPARTMENT a ON x.DeptID = a.DeptID
JOIN EMPLOYEE b ON x.EmpID = b.EmpID;
```

2(b) Aggregate query: Group the transaction rows by the requested category/type and apply HAVING to the aggregate condition stated in the question.

```
SELECT grouping_column, COUNT(*) AS total
FROM transaction_table
GROUP BY grouping_column
HAVING COUNT(*) > threshold;
```

2(c) PL/SQL procedure: Create a procedure with an IN parameter for the requested ID. Join the transaction table to the related entity tables, filter by that parameter, and print selected values using DBMS_OUTPUT.PUT_LINE.

```
CREATE OR REPLACE PROCEDURE show_details(p_id IN NUMBER) AS
BEGIN
  FOR r IN (
    SELECT a.*, b.*, x.*
    FROM transaction_table x
    JOIN entity_table_1 a ON x.entity1_id = a.entity1_id
    JOIN entity_table_2 b ON x.entity2_id = b.entity2_id
    WHERE x.transaction_id = p_id
  ) LOOP
    DBMS_OUTPUT.PUT_LINE('Details retrieved');
  END LOOP;
END;
/
```

3. MongoDB: Create the collections and insert documents using the fields listed in the set. Query with find() and the requested filter.

```
db.collection.insertMany([
  { _id: 1, name: 'Sample A', category: 'Example', price: 500 },
  { _id: 2, name: 'Sample B', category: 'Example', price: 300 }
]);
db.collection.find({ author: 'Specified Author' });
db.collection.find({ price: { $gt: 400 } });
```